PROJECT REPORT

TITLE

AI-Based Student Help Chatbot (CLI Application)

INTRODUCTION

In this project, I developed a simple chatbot using basic Artificial Intelligence concepts. The chatbot is designed to interact with users through a command-line interface and provide helpful guidance to students. Instead of using complex AI models, this chatbot works on a rule-based system, making it easy to understand and implement.

MOTIVATION

As a student, I have noticed that many students face common problems like exam stress, poor time management, and confusion about studies. Searching for solutions every time can be time-consuming. So, I thought of creating a simple chatbot that can quickly provide useful suggestions and guidance.

PROBLEM STATEMENT

The aim of this project is to design a command-line chatbot that can answer basic student-related queries using predefined rules and keywords.

OBJECTIVES

- To build a chatbot using Python
- To understand and apply rule-based AI concepts
- To provide simple and helpful responses to students
- To create a complete CLI-based application

EXISTING METHODS

Most chatbots today use machine learning and natural language processing. These methods require large datasets and are more complex to build. In comparison, this project uses a simple rule-based approach which is easier to implement.

PROS AND CONS

Pros:

- Easy to design and understand
- Fast response time
- Does not require any training data

Cons:

- Can only respond to predefined inputs
- Cannot understand complex or unexpected questions

REQUIREMENTS

Software:

- Python 3
- Command Prompt / Terminal

Hardware:

- Basic computer or laptop

METHODOLOGY

In this project, the chatbot works by matching user input with predefined keywords such as "exam", "stress", "time", etc. Based on the keyword detected, it returns a suitable response. I also added a feature to store chat history in a JSON file so that previous conversations can be saved.

FUNCTIONAL MODULES

- User Input Module – Takes input from the user
- Response Matching Module – Checks keywords and selects response
- Chat History Module – Stores conversation in a JSON file
- Menu System – Allows user to interact with the program easily

ALGORITHM

1. Start the program
2. Display menu options
3. Take input from user

4. Check for matching keywords
5. Display the response
6. Save the chat in history file
7. Repeat until the user exits

IMPLEMENTATION

The chatbot is implemented using Python. I used dictionaries to store keywords and their corresponding responses. The chat history is stored using a JSON file (for example, history.json). The entire program runs in the terminal, making it lightweight and easy to use.

OUTPUT

The chatbot is able to respond to common student queries related to exams, stress, and time management. It gives quick and simple suggestions based on user input.

RESULTS

The chatbot works well for predefined inputs and provides useful guidance. It performs efficiently and responds instantly to user queries.

CHALLENGES FACED

While developing this project, I faced some difficulties in deciding the correct keywords for user queries. Sometimes the chatbot was not giving the expected response if the input did not match exactly. I also had to ensure that the chat history was stored properly in the JSON file.

LIMITATIONS

- Works only for specific keywords
- Cannot understand long or complex sentences
- No learning capability

FUTURE ENHANCEMENTS

- Add machine learning for better understanding
- Improve response quality using NLP
- Develop a voice-based chatbot

CONCLUSION

Through this project, I understood how basic AI concepts like rule-based systems work in real-life applications. It helped me learn how chatbots process input and generate responses. Overall, this project was useful in understanding simple AI implementation using Python.